Plant Light - Design Documentation

Student: Briyon Benny
Student ID: 3144036
Module: BSC-MD Developing an Android App
Milestone: Milestone 1
GitHub repository: https://github.com/briyonbenny/PlantLight.git

1 Main idea

Many people buy a houseplant without knowing whether the spot they have in mind gets enough light, and the plant struggles as a result. Plant Light is an Android app that helps people choose the right spot in their home for a houseplant. The user holds the phone at a windowsill, shelf or desk; the app reads the phone's ambient light sensor, averages the lux value over 3 seconds, and classifies the spot as Low (below 200 lux), Medium (200–1,000 lux) or Bright (1,000 lux and above). The reading can be saved under a name, and the app suggests houseplants from a built-in plant catalogue that suit that light level. Saved spots can be photographed and shared. If a device has no light sensor, the user picks Low, Medium or Bright by hand instead. All data stays on the phone; nothing is uploaded.

2 Objectives

- **Measure:** Measure how bright a spot is with the ambient light sensor, using a 3-second average so the result is stable, and show it on a clear live gauge.
- **Classify:** Turn the lux value into a Low / Medium / Bright label using fixed thresholds (200 and 1,000 lux), with manual selection as a fallback when there is no sensor.
- **Save and manage:** Let the user name and save spots, browse them in a list, open their details and delete them.
- **Recommend:** Suggest suitable houseplants for each spot from a catalogue stored on the device.
- **Photo and share:** Attach a photo of the spot (system camera) and share a short text report with the photo (system share sheet).
- **Private and complete:** Keep everything local and private (Room, DataStore, app-private files) and meet all Milestone 1 requirements: three distinctive layouts, user interaction, and explicit and implicit intents.

3 Design

3.1 Screens and navigation

The app has three main activities plus an optional Settings screen. Measure is the launcher screen; its toolbar has a My Spots icon and a Settings icon. Navigation: Measure opens My Spots (button or toolbar icon); tapping a row in My Spots opens Spot Details with the spot ID (explicit intent); after saving on Measure, a snackbar "Saved …" offers a View action that also opens Spot Details; Spot Details opens the system camera and share sheet (implicit intents); the Settings icon on Measure opens Settings.

- **Measure –** A custom gauge (LightGaugeView) showing the live averaged reading and its level, a name field, and Save and My Spots buttons. Without a sensor, Low / Medium / Bright chips appear instead. Saving shows a snackbar with a View action.
- **My Spots –** A RecyclerView of saved spots; each item shows the name, level badge, reading and a photo thumbnail, plus a delete icon with a confirmation dialog. An empty state ("No spots yet") appears when nothing is saved.
- **Spot Details –** The spot's name, level and reading, the suggested plants for that level, the photo, and Take photo and Share buttons.
- **Settings (optional) –** Display units (lux or foot-candles, saved in DataStore) and a Clear data button that removes all spots and photos after a confirmation but keeps the plant catalogue.

3.2 How the Milestone 1 requirements are met

Requirement	Where it is met in Plant Light

Three distinctive functional layouts	Measure (custom gauge and form), My Spots (RecyclerView list), Spot Details (detail view with plants, photo and actions); Settings is an extra fourth screen.
User interaction	Live gauge or manual chips, naming and saving a spot, snackbar View action, opening and deleting spots, taking a photo, sharing, changing units, clearing data.
Explicit intent	PlantLightIntents.spotDetails(context, spotId) starts SpotDetailsActivity and passes EXTRA_SPOT_ID (from My Spots and from the snackbar on Measure).
Implicit intents	ACTION_IMAGE_CAPTURE: the system camera writes to a FileProvider URI in the app-private photos folder. ACTION_SEND: a chooser shares a text report (and the photo when one exists).

3.3 Data and architecture

The app is written in Kotlin with XML layouts and view binding (no Jetpack Compose). A PlantLightApp Application class holds one Room database, the DataStore settings and the photo store, so every activity uses the same local data. Database and file work runs in coroutines (lifecycleScope), keeping the UI responsive.

- **Room database (plant_light.db):** spots (id, name, lightLevel, averageLux, isManual, createdAt, photoPath); light_readings (spotId linked to spots with cascade delete, lux, timestamp), ready for future day-long tracking; plants (name, lightNeed, notes), seeded on first launch with 18 houseplants, six per light level (e.g. Snake Plant, Monstera, Aloe Vera). Room uses KSP for code generation.
- **Settings:** Jetpack DataStore stores the display unit (lux or foot-candles; 1 fc ≈ 10.764 lux).
- **Photos:** PhotoStore saves photos in the app's private files/photos/ folder, exposed only through a FileProvider URI for the camera and share intents; photos are deleted with their spot.
- **Logic:** LightClassifier holds the thresholds and a 3-second LuxAverager; LightLevel is the Low / Medium / Bright enum. minSdk 24, target/compile SDK 36.

3.4 UI design

The app uses a calm, plant-inspired Material 3 theme with green toolbars and buttons on a light cream background. Each light level has its own colour (sage for Low, gold for Medium, terracotta for Bright), used consistently on the gauge and the list badges so the level can be read at a glance. Cards, text fields, chips, snackbars and confirmation dialogs are standard Material components, so the app feels familiar, and destructive actions (delete, clear data) always ask first. The gauge is a custom View because a needle moving across zones is easier to interpret than a plain number.

4 Next milestones

- Day-long light tracking (morning / midday / evening) for a spot, building on the existing light_readings table.
- Sensor calibration, extending the DataStore settings.

Both features were left out of Milestone 1 on purpose, but the database and settings were designed so they can be added without changing the existing structure.